

Fiche 310 — Portée lexicale, environnements et état mutable

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — §10.7 « Scope », §10.8 « Customizing the environment »
Sources d'appoint	<i>R Language Definition</i> 4.6.1, §3.5 « Scope of variables » (§3.5.1 environnement global, §3.5.2 environnement lexical, §3.5.3 la pile d'appels , §3.5.4 chemin de recherche), §4.3.4 « Scope », §2.1.10 « Environments », §2.1.5 « Function objects »
Difficulté	Avancé — « <i>one of the major differences between S-Plus and R</i> »
Temps d'étude estimé	1 h 30
Prérequis	Fiches 303, 306, 309 (attributs, chemin de recherche, fonctions)
Concepts clés	paramètre formel, variable locale , variable libre , portée lexicale contre statique , fermeture (<i>closure</i>), <i>frame</i> contre <i>environment</i> , enclos , <code><-</code> , état mutable , <code>environment()</code> , portée dynamique et pile d'appels, famille <code>sys.*</code> , <code>parent.frame()</code> , chemin de recherche et espaces de noms , <code>.First()</code> et <code>.Last()</code> , <code>Rprofile.site</code> , <code>.Rprofile</code>
À retenir en priorité	Les trois classes de symboles · portée lexicale = l'environnement de <i>création</i> , pas celui d'appel · <code><-</code> remonte les enclos · le fait qu'un environnement n'est pas copié.

Vue d'ensemble

TROIS CLASSES DE SYMBOLES dans le corps d'une fonction

PARAMETRES FORMELS ceux de la liste d'arguments

VARIABLES LOCALES définies par une assignation dans le corps

VARIABLES LIBRES ni l'un ni l'autre

« Free variables BECOME LOCAL VARIABLES IF THEY ARE ASSIGNED TO. »

LA REGLE

« In R the free variable bindings are resolved by FIRST LOOKING IN THE ENVIRONMENT IN WHICH THE FUNCTION WAS CREATED. This is called LEXICAL SCOPE. »

	S-Plus (portee STATIQUE) -> cherche une variable GLOBALE R (portee LEXICALE) -> cherche la ou la fonction est NEE
VOCABULAIRE	un FRAME = un ensemble de variables un ENVIRONMENT = un emboitement de frames = frame le plus interne + ENCLOS
UNE FERMETURE	code + environnement de creation « NOTICE THAT THIS IS NOT NECESSARILY THE ENVIRONMENT OF THE CALLER! »
LA RECHERCHE	environnement d'evaluation -> enclos -> enclos de l'enclos -> environnement global ou d'un paquet -> chemin de recherche -> base -> environnement VIDE -> echec
<<-	remonte les ENCLOS jusqu'a trouver le symbole et REMPLACE la valeur LA-BAS s'il n'existe nulle part -> CREE au niveau global
DEMARRAGE	Rprofile.site -> profil utilisateur -> .RData -> .First() « A definition in later files will MASK definitions in earlier files. »

Le problème posé. « *The discussion in this section is **somewhat more technical** than in other parts of this document. **However, it details one of the major differences between S-Plus and R.*** » (\$10.7)

Le cours prévient rarement de la sorte. Ce paragraphe n'est pas un raffinement : c'est **ce qui distingue R de son modèle**, et ce qui rend possibles des constructions impossibles ailleurs.

● Concept 1 — Trois classes de symboles

DÉFINITION (§10.7).

« *The symbols which occur in the body of a function can be divided into three classes : formal parameters, local variables and free variables.* »

- « *The formal parameters of a function are those occurring in the argument list. Their values are determined by the process of binding the actual function arguments to the formal parameters.* »
- « *Local variables are those whose values are determined by the evaluation of expressions in the body of the function.* »
- « *Variables which are neither formal parameters nor local variables are called free variables. Free variables become local variables if they are assigned to.* »

```
f <- function(x) {  
  y <- 2*x  
  print(x)  
  print(y)  
  print(z)  
}
```

« In this function, `x` is a formal parameter, `y` is a local variable and `z` is a free variable. »

La dernière phrase de la définition est la plus lourde de conséquences. « *Free variables become local variables if they are assigned to.* »

Une variable n'est pas libre ou locale **par nature** : elle l'est **selon ce que fait le corps**. Ajouter quelque part un `z <- 0` transforme `z` de libre en locale, **dans toute la fonction** — y compris avant la ligne d'assignation. Le comportement de la fonction change alors du tout au tout, et rien dans le message d'erreur ne le signale.

COMPLÉMENT (R LANGUAGE DEFINITION §4.3.4).

« *Because R has no formal definition of variables, they are simply used as needed, it can be difficult to determine whether a variable is local or not. Local variables must first be defined, this is typically done by having them on the left-hand side of an assignment.* »

● Concept 2 — Portée lexicale contre portée statique

Règle (§10.7). « *In R the free variable bindings are resolved by first looking in the environment in which the function was created. This is called lexical scope.* »

L'exemple qui prouve la différence (§10.7) :

```
cube <- function(n) {  
  sq <- function() n*n  
  n*sq()  
}
```

*« The variable `n` in the function `sq` is not an argument to that function. Therefore it is a free variable and the scoping rules must be used to ascertain the value that is to be associated with it. **Under static scope (S-Plus) the value is that associated with a global variable named `n`.** Under lexical scope (R) it is the parameter to the function `cube`, since that is the active binding for the variable `n` at the time the function `sq` was defined. »*

Le résultat, tel que le cours le montre :

```
## d'abord dans S  
S> cube(2)  
Error in sq(): Object "n" not found  
Dumped  
S> n <- 3  
S> cube(2)  
[1] 18  
  
## puis la meme fonction dans R
```

```
R> cube(2)
```

```
[1] 8
```

CONTRÔLE — RETROUVER LES DEUX NOMBRES.

Sous **R** : `sq()` voit `n = 2` (le paramètre de `cube`), donc `sq()` vaut **4** et `cube(2)` vaut $2 \times 4 = 8$. Sous **S-Plus** avec `n <- 3` en global : `sq()` voit `n = 3`, donc `sq()` vaut **9**, et `n*sq()` utilise... le **n local**, soit **2** — d'où $2 \times 9 = 18$.

Le même code, deux résultats, et aucun message. C'est pourquoi le cours qualifie ce paragraphe de « *one of the major differences* ».

« The difference ... is that **S-Plus looks for a global variable called `n`** while **R first looks for a variable called `n` in the environment created when `cube` was invoked.** »

COMPLÉMENT — LA RÈGLE COMPLÈTE (R LANGUAGE DEFINITION §3.5.2).

« when a variable is requested inside a function, it is **first sought in the evaluation environment**, then in **the enclosure, the enclosure of the enclosure**, etc. ; once **the global environment or the environment of a package** is reached, the search continues **up the search path** to the environment of **the base package**. If the variable is not found there, the search will proceed next to **the empty environment, and will fail.** »

● Concept 3 — Frame, environnement, fermeture

DÉFINITION (R LANGUAGE DEFINITION §3.5.2).

« **Every call to a function creates a frame which contains the local variables created in the function, and is evaluated in an environment, which in combination creates a new environment.** »

« **Notice the terminology : a frame is a set of variables, an environment is a nesting of frames (or equivalently : the innermost frame plus the enclosing environment).** »

Terme	Définition
Frame	un ensemble de variables — des paires symbole-valeur
Environment	un emboîtement de frames = le frame le plus interne + l'enclos
Enclosure	le pointeur vers l'environnement englobant

DÉFINITION — LA FERMETURE (§3.5.2).

« A closure (mode "function") object will contain the environment in which it is created as part of its definition (by default ; the environment can be manipulated using `environment<-`). When the function is subsequently called, its evaluation environment is created with the closure's environment as enclosure. Notice that this is not necessarily the environment of the caller! »

C'est la phrase qui résume toute la fiche. Une fonction ne cherche **pas** ses variables libres là où on l'appelle : elle les cherche **là où elle est née**.

COMPLÉMENT (§2.1.5, FICHE 303).

*« Functions (or more precisely, **function closures**) have **three basic components** : a **formal argument list**, a **body** and an **environment**. ... A function's environment is the environment that was active at the time that the function was created. Any symbols bound in that environment are captured and available to the function. This combination of the code of the function and the bindings in its environment is called a "function closure", a term from functional programming theory. »*

« It is possible to **extract and manipulate the three parts** of a closure object using `formals` , `body` , and `environment` (all three can also be used on the left hand side of assignments). The last of these can be used to remove unwanted environment capture. »

Et le rappel décisif (§3.5.2). « Environments may be assigned to variables or be contained in other objects. **However, notice that they are not standard objects — in particular, they are not copied on assignment.** » — c'est l'unique exception à la sémantique de copie (fiches 302, 303, 309).

● Concept 4 — `<<-` et l'état mutable

DÉFINITION (§10.7).

« *The special assignment operator, `<<-`, is used to change the value associated with `total`. This operator looks back in enclosing environments for an environment that contains the symbol `total`, and when it finds such an environment it replaces the value, in that environment, with the value of right hand side. If the global or top-level environment is reached without finding the symbol `total` then that variable is created and assigned to there.* »

Les deux comportements, à ne pas confondre :

Situation	Ce que fait <code><<-</code>
Le symbole existe dans un enclos	il remplace sa valeur là-bas
Le symbole n'existe nulle part	il le crée au niveau global

« *For most users `<<-` creates a global variable and assigns the value of the right hand side to it. Only when `<<-` has been used in a function that was returned as the value of another function will the special behavior described here occur.* » (§10.7)

Autrement dit : dans une fonction ordinaire, `<<-` **pollue l'espace de travail**. Son intérêt véritable apparaît **uniquement** dans une fonction **renvoyée par une autre** — la construction du concept suivant.

(Note 2 du §10.7 : « *In some sense this mimics the behavior in S-Plus since in S-Plus this operator **always** creates or assigns to a global variable.* »)

4.1 Le compte en banque — l'exemple complet du cours

Énoncé (\$10.7). « *Lexical scope can also be used to give functions mutable state. In the following example we show how R can be used to mimic a bank account. A functioning bank account needs to have **a balance or total**, **a function for making withdrawals**, **a function for making deposits** and **a function for stating the current balance**. We achieve this by creating the three functions within `account` and then returning a list containing them. ... Because these functions are defined in an environment which contains `total`, they will have access to its value.* »

```
open.account <- function(total) {  
  list(  
    deposit = function(amount) {  
      if (amount <= 0)  
        stop("Deposits must be positive!\n")  
      total <- total + amount  
      cat(amount, "deposited. Your balance is", total, "\n\n")  
    },  
    withdraw = function(amount) {  
      if (amount > total)  
        stop("You don't have that much money!\n")  
      total <- total - amount  
      cat(amount, "withdrawn. Your balance is", total, "\n\n")  
    },  
    balance = function() {  
      cat("Your balance is", total, "\n\n")  
    }  
  )  
}  
  
ross <- open.account(100)  
robert <- open.account(200)  
  
ross$withdraw(30)  
ross$balance()  
robert$balance()  
  
ross$deposit(50)
```



```
ross$balance()  
ross$withdraw(500)
```

Étape 1 — ce que crée `open.account(100)`. L'appel crée **un environnement d'évaluation** contenant le paramètre formel `total = 100`. Les trois fonctions y sont **définies**, donc chacune **capture cet environnement** comme le sien (§2.1.5 : « *a function's environment is the environment that was active at the time that the function was created* »). Le résultat est **une liste** de trois fermetures.

Étape 2 — pourquoi `total` survit à la fin de `open.account`. L'appel est terminé, mais **les trois fonctions renvoyées pointent encore vers son environnement**. Tant qu'elles existent, **il existe**. C'est ce que le cours appelle « *mutable state* ».

Étape 3 — deux comptes indépendants. `robert <- open.account(200)` déclenche **un second appel**, donc **un second environnement**, avec son propre `total`. Les deux jeux de fermetures ne se voient pas.

Étape 4 — `ross$withdraw(30)`. Le garde-fou `if (amount > total) stop(...)` : **30 ≤ 100**, on passe. Puis `total <- total - amount`. Le `<-` « *looks back in enclosing environments for an environment that contains the symbol total* » — et il le trouve **au premier enclos**, celui de l'appel à `open.account`. **Le solde de ross passe à 70**. Affichage : « 30 withdrawn. Your balance is 70 ».

Étape 5 — `ross$balance()` puis `robert$balance()`. **70** et **200**. La preuve que les deux environnements sont distincts.

Étape 6 — `ross$deposit(50)`. **50 > 0**, on passe. `total <- total + 50` → **120**.

Étape 7 — `ross$withdraw(500)`. **500 > 120** : `stop("You don't have that much money!\n")`. L'exécution est **interrompue** — et, point important, `total` **n'a pas bougé** : le `stop` précède l'assignation. C'est pourquoi le garde-fou est écrit **en premier**.

Étape 8 — pourquoi `<-` ne marcherait pas. Avec `total <- total - amount`, l'assignation serait **locale à withdraw** (fiche 309, §10.5 : « *local and temporary and lost after exit* »). Le solde reviendrait à sa valeur initiale au retrait suivant. `<-` **est ce qui rend l'état persistant**.

Étape 9 — la leçon. Ce mécanisme est **l'objet, sans système d'objets** : des données privées (`total`), une interface publique (les trois fonctions), et **aucun moyen d'accéder au solde autrement que par elles**. C'est ce que la fiche 311 comparera à S3.

● Concept 5 — La portée dynamique et la pile d'appels

DÉFINITION (R LANGUAGE DEFINITION §3.5.3).

« **Every time a function is invoked a new evaluation frame is created.** At any point in time during the computation **the currently active environments are accessible through the call stack.** Each time a function is invoked **a special construct called a context is created internally and is placed on a list of contexts.** When a function has finished evaluating its context is removed from the call stack. »

DÉFINITION — L'AUTRE PORTÉE (§3.5.3).

« **Making variables defined higher up the call stack available is called dynamic scope.** The binding for a variable is then determined by **the most recent (in time) definition** of the variable. **This contradicts the default scoping rules in R, which use the bindings in the environment in which the function was defined (lexical scope).** Some functions, particularly those that use and manipulate model formulas, need to simulate dynamic scope by directly accessing the call stack. »

	Portée lexicale (le défaut)	Portée dynamique (simulée)
Où l'on cherche	l'environnement de définition	la pile d'appels
Ce qui décide	le lieu d'écriture	le moment de l'appel
Qui l'utilise	tout R	les fonctions de formules de modèle

C'est l'explication technique du comportement de `lm(y ~ x, data = dummy)` (fiche 313) : la formule `y ~ x` doit être évaluée **dans le contexte de l'appelant**, pas dans celui de `lm`.

NOTATION (§3.5.3).

« Access to the call stack is provided through **a family of functions which have names that start with `sys.`** » :

Fonction	Rôle
<code>sys.call</code>	« Get the call for the specified context »
<code>sys.frame</code>	« Get the evaluation frame for the specified context »
<code>sys.nframe</code>	« Get the environment frame for all active contexts »
<code>sys.function</code>	« Get the function being invoked in the specified context »
<code>sys.parent</code>	« Get the parent of the current function invocation »
<code>sys.calls</code> / <code>sys.frames</code> / <code>sys.parents</code>	les mêmes, pour tous les contextes actifs
<code>sys.on.exit</code>	« Set a function to be executed when the specified context is exited »
<code>sys.status</code>	« Calls <code>sys.frames</code> , <code>sys.parents</code> and <code>sys.calls</code> »
<code>parent.frame</code>	« Get the evaluation frame for the specified parent context »

● Concept 6 — Le chemin de recherche et les espaces de noms

DÉFINITION (R LANGUAGE DEFINITION §3.5.1).

« *The global environment is the root of the user workspace. An assignment operation from the command line will cause the relevant object to belong to the global environment. Its enclosing environment is the next environment on the search path, and so on back to the empty environment that is the enclosure of the base environment.* »

DÉFINITION (§3.5.4).

« *In addition to the evaluation environment structure, **R** has a search path of environments which are searched for variables not found elsewhere. This is used for two things : packages of functions and attached user data.* »

« *The first element of the search path is the global environment and the last is the base package. An `AutoLoads` environment is used for holding proxy objects that may be loaded on demand. Other environments are inserted in the path using `attach` or `Library`.* »

C'est exactement ce que montrait `search()` en fiche 306 : `.GlobalEnv` , `AutoLoads` , `package:base` .

L'exception des espaces de noms (§3.5.4). « *Packages which have a namespace have a different search path. When a search for an R object is started from an object in such a package, the package itself is searched first, then its imports, then the base namespace and finally the global environment and the rest of the regular search path. The effect is that references to other objects in the same package will be resolved to the package, and objects cannot be masked by objects of the same name in the global environment or in other packages.* »

C'est la solution du problème de masquage rencontré en fiche 306. Un paquet à espace de noms est immunisé : redéfinir `mean` dans votre espace de travail **ne change rien** au comportement interne du paquet. Voir fiches 315 et 319.

● Concept 7 — Personnaliser son environnement

Règle (§10.8). « *Users can customize their environment in several different ways. There is a site initialization file and every directory can have its own special initialization file. Finally, the special functions `.First` and `.Last` can be used.* »

La cascade, telle que le §10.8 la décrit — la même que celle de l'annexe B (fiche 300), vue cette fois du côté de l'utilisateur :

Fichier	Où R le cherche
Profil de site	la valeur de <code>R_PROFILE</code> ; si non défini, <code>Rprofile.site</code> dans le sous-répertoire <code>etc</code> du répertoire d'installation. « <i>This file should contain the commands that you want to execute every time R is started under your system</i> »
Profil personnel	<code>.Rprofile</code> — « <i>can be placed in any directory. If R is invoked in that directory then that file will be sourced.</i> » Sinon, R en cherche un dans le répertoire personnel. Si <code>R_PROFILE_USER</code> est défini, c'est ce fichier qui est utilisé à la place

« *This file gives individual users control over their workspace and allows for different startup procedures in different working directories.* » (note 3 : « *So it is hidden under UNIX* ».)

DÉFINITION — `.FIRST()` (§10.8).

« *Any function named `.First()` in either of the two profile files or in the `.RData` image has a special status. It is automatically performed at the beginning of an R session and may be used to initialize the environment.* »

La règle d'ordre, et sa conséquence (§10.8). *« Thus, the sequence in which files are executed is : `Rprofile.site` , the user profile, `.RData` and then `.First()` . A definition in later files will mask definitions in earlier files. »*

C'est la séquence de la fiche 300, avec la précision qui manquait : **le dernier chargé l'emporte**. Un `.RData` peut donc écraser une définition du profil de site — et le faire **silencieusement**.

L'exemple du cours :

```
.First <- function() {  
  options(prompt = "$ ", continue = "+\t")    # $ est l'invite  
  options(digits = 5, length = 999)          # nombres et affichage  
  x11()                                       # pour les graphiques  
  par(pch = "+")                             # caractere de trace  
  source(file.path(Sys.getenv("HOME"), "R", "mystuff.R"))  
                                              # mes fonctions personnelles  
  library(MASS)                             # attacher un paquet  
}
```

« Similarly **a function** `.Last()` , if defined, is (normally) executed at the very end of the session » :

```
.Last <- function() {                      # a small safety measure.  
  graphics.off()                          # Is it time for lunch?  
  cat(paste(date(), "\nAdios\n"))  
}
```

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« d'où vient la valeur de cette variable ? »	classer le symbole : formel , local , libre
« le même code donne un autre résultat en S »	portée lexicale contre statique
« la fonction interne voit-elle <code>n</code> ? »	oui, si <code>n</code> existait là où elle a été définie
« ajouter une ligne casse tout »	« <i>free variables become local variables if they are assigned to</i> »
« garder un état entre deux appels »	une fermeture + <code><<-</code>
« deux compteurs indépendants »	deux appels = deux environnements
« <code><<-</code> crée une variable globale »	c'est le cas hors fermeture
« mon objet est modifié à distance »	c'est un environnement — non copié
« la fonction doit voir le contexte de l'appelant »	portée dynamique → <code>parent.frame()</code> , famille <code>sys.*</code>
« pourquoi <code>lm</code> trouve-t-il mes colonnes ? »	il simule la portée dynamique
« redéfinir <code>mean</code> casse-t-il un paquet ? »	non , s'il a un espace de noms
« exécuter du code au démarrage »	<code>.Rprofile</code> et <code>.First()</code>
« ma définition est écrasée »	« <i>a definition in later files will mask definitions in earlier files</i> »

Comment résoudre ce type d'exercice

Protocole « d'où vient cette valeur ? » — 4 étapes.

1. **Classer le symbole** : est-il dans la liste d'arguments (**formel**) ? assigné dans le corps (**local**) ? ni l'un ni l'autre (**libre**) ?
2. Si libre, **remonter les enclos** — « *first sought in the evaluation environment, then in the enclosure, the enclosure of the enclosure* ».
3. Puis **le chemin de recherche** jusqu'à `base` , puis **l'environnement vide** → échec.
4. Se rappeler que **l'enclos est l'environnement de définition, pas celui d'appel**.

Protocole « fabriquer un état mutable » — 4 étapes.

1. Écrire **une fonction fabrique** dont les paramètres portent l'état.
2. **Définir à l'intérieur** les fonctions qui manipuleront cet état.
3. Utiliser `<<-` pour l'écrire — jamais `<-`, qui resterait local.
4. **Renvoyer une liste** de ces fonctions. Chaque appel de la fabrique produit **un état indépendant**.

Protocole « diagnostiquer un masquage » — 4 étapes.

1. `search()` pour l'ordre des positions, `ls(n)` pour leur contenu (fiche 306).
2. Vérifier si le paquet a **un espace de noms** — auquel cas son fonctionnement interne est protégé.
3. Devant un comportement de démarrage inexplicable : `--vanilla` (fiche 300), puis remonter la cascade `Rprofile.site` → profil utilisateur → `.RData` → `.First()`.
4. Ne pas oublier que `.RData` **passe après les profils** et peut donc les écraser.

● Common mistakes

Erreur	Correction
Croire qu'une fonction voit les variables de l'appelant	« <i>not necessarily the environment of the caller!</i> »
Croire une variable définitivement libre	elle devient locale si on l'assigne quelque part
Porter du code S-Plus sans vérifier la portée	le même <code>cube(2)</code> rend 8 en R, 18 en S
Utiliser <code><-</code> pour maintenir un état	il est local et perdu à la sortie
Utiliser <code><<-</code> dans une fonction ordinaire	il crée une variable globale
Croire <code><<-</code> toujours global	dans une fermeture, il écrit dans l'enclos
Assigner avant le garde-fou	mettre <code>stop()</code> en premier
Croire un environnement copié à l'assignation	« <i>they are not copied on assignment</i> »
Confondre <i>frame</i> et <i>environment</i>	frame = des variables ; environment = frame + enclos
Attendre la portée dynamique par défaut	R est lexical ; la dynamique se simule
Croire qu'on peut masquer une fonction interne d'un paquet	pas s'il a un espace de noms
Croire le profil de site prioritaire	« <i>a definition in later files will mask</i> » — <code>.RData</code> passe après
Oublier que <code>.First()</code> peut venir de <code>.RData</code>	et que <code>ls()</code> ne montre pas les noms à point

Ultimate Review

Trois classes de symboles. Formels — ceux de la liste d'arguments, liés au moment de l'appel · **locaux** — « *whose values are determined by the evaluation of expressions in the body* » · **libres** — ni l'un ni l'autre. « *Free variables become local variables if they are assigned to.* » Et : « *Local variables must first be defined, typically by having them on the left-hand side of an assignment* ».

La règle. « *In R the free variable bindings are resolved by first looking in the environment in which the function was created. This is called lexical scope.* » S-Plus, en portée **statique**, cherche **une variable globale**. D'où `cube(2)` = **8** en R et **18** en S-Plus après `n <- 3`.

La chaîne de recherche. environnement d'évaluation → **enclos** → enclos de l'enclos → environnement global ou de paquet → chemin de recherche → base → environnement vide → échec.

Vocabulaire. « *A frame is a set of variables, an environment is a nesting of frames (or equivalently : the innermost frame plus the enclosing environment).* »

La fermeture. « *A closure ... will contain the environment in which it is created as part of its definition* » — modifiable par `environment<-` · à l'appel, « *its evaluation environment is created with the closure's environment as enclosure* » · « *Notice that this is not necessarily the environment of the caller!* » · trois composantes accessibles par `formals` , `body` , `environment` , toutes trois **utilisables à gauche d'une assignation** · les environnements **ne sont pas copiés**.

`<<-` . « *looks back in enclosing environments for an environment that contains the symbol ... and replaces the value, in that environment* » · si le symbole n'est trouvé nulle part, il est **créé au niveau global** · « *For most users <<- creates a global variable ... Only when <<- has been used in a function that was returned as the value of another function will the special behavior ... occur.* »

L'état mutable. « *Lexical scope can also be used to give functions mutable state.* » Le patron : une **fabrique** dont le paramètre porte l'état, des fonctions **définies dedans**, `<<-` pour écrire, une **liste** en retour. Chaque appel de la fabrique crée **un état indépendant** — `ross` et `robert` ne se voient pas. Et **le garde-fou avant l'assignation** : `ross$withdraw(500)` s'arrête sans toucher au solde.

Portée dynamique. « *Making variables defined higher up the call stack available is called dynamic scope ... determined by the most recent (in time) definition ... This contradicts the default scoping rules in R ... Some functions, particularly those that use and manipulate model formulas, need to simulate dynamic scope by directly accessing the call stack.* » Outils : la famille `sys.*` (`sys.call` , `sys.frame` , `sys.function` , `sys.parent` , `sys.calls` , `sys.frames` , `sys.parents` , `sys.on.exit` , `sys.status`) et `parent.frame` .

Chemin de recherche. « *The global environment is the root of the user workspace ... Its enclosing environment is the next environment on the search path ... back to the empty environment* » · premier élément `.GlobalEnv` , dernier `base` , plus `AutoLoads` pour les objets chargés à la demande · on insère avec `attach` ou `library` .

Espaces de noms. « *Packages which have a namespace have a different search path : the package itself, then its imports, then the base namespace and finally the global environment ... objects cannot be masked by objects of the same name in the global environment or in other packages.* »

Personnalisation. `Rprofile.site` (via `R_PROFILE` , sinon `etc/Rprofile.site`) → **profil utilisateur** (`.Rprofile` du répertoire courant, sinon du répertoire personnel, sinon `R_PROFILE_USER`) → `.RData` → `.First()` . « *A definition in later files will mask definitions in earlier files.* » Et `.Last()` « *is (normally) executed at the very end of the session* ».

Active Recall

« *The symbols which occur in the body of a function can be divided into three classes : formal parameters, local variables and free variables.* » (§10.7)

- **Formels** : ceux de la liste d'arguments ; « *their values are determined by **the process of binding the actual function arguments*** ».
- **Locaux** : « *those whose values are determined by the evaluation of expressions in the body* ».
- **Libres** : « *variables which are **neither** formal parameters **nor** local variables* ».

La distinction n'est pas figée : « *Free variables become local variables if they are assigned to.* » Ajouter un `z <- 0` n'importe où dans le corps rend `z` **locale partout** — y compris avant la ligne d'assignation. Le comportement change du tout au tout, sans message.

Et (§4.3.4) : « *Because R has **no formal definition of variables**, they are simply used as needed, **it can be difficult to determine whether a variable is local or not*** ».

« *In R the free variable bindings are resolved by first looking in the environment in which the function was created. This is called lexical scope.* » (§10.7)

```
cube <- function(n) {  
  sq <- function() n*n  
  n*sq()  
}
```

*« Under **static scope (S-Plus)** the value is **that associated with a global variable named `n`**. Under **lexical scope (R)** it is **the parameter to the function `cube`**, since that is **the active binding for the variable `n` at the time the function `sq` was defined.** »*

En R : `sq()` voit `n = 2` → $2 \times 4 = 8$. **En S-Plus** avec `n <- 3` en global : `sq()` voit `n = 3` → 9, et `n*sq()` utilise le `n` local, $2 \rightarrow 2 \times 9 = 18$.

Et sans `n` global, S-Plus **échoue** : `Error in sq(): Object "n" not found` .

« **Notice the terminology : a frame is a set of variables, an environment is a nesting of frames** (or equivalently : **the innermost frame plus the enclosing environment**). » (R Language Definition §3.5.2)

Un **frame** est plat : des paires symbole-valeur. Un **environnement** est **chaîné** : un frame **plus un pointeur** vers l'environnement englobant — l'**enclos**.

C'est ce qui donne la chaîne de recherche du §3.5.2 : « *first sought in **the evaluation environment**, then in **the enclosure, the enclosure of the enclosure**, etc.* »

Le §2.1.10 (fiche 303) le disait déjà : « *Environments can be thought of as consisting of two things : **a frame** ... and **an enclosure**, a pointer to an enclosing environment.* »

« **A closure (mode "function") object will contain the environment in which it is created as part of its definition** ... When the function is subsequently called, **its evaluation environment is created with the closure's environment as enclosure.** » (§3.5.2)

La phrase : « **Notice that this is not necessarily the environment of the caller!** »

Une fonction cherche ses variables libres **là où elle est née**, pas **là où on l'appelle**. C'est toute la différence entre portée lexicale et portée dynamique.

Et (§2.1.5) : « *Functions ... have **three basic components : a formal argument list, a body and an environment** ... This combination of the code of the function and the bindings in its environment is called a "function closure"* », accessibles par `formals` , `body` , `environment` — « **all three can also be used on the left hand side of assignments** ».

« **This operator looks back in enclosing environments for an environment that contains the symbol `total` , and when it finds such an environment it replaces the value, in that environment, with the value of right hand side. If the global or top-level environment is reached without finding the symbol then that variable is created and assigned to there.** » (§10.7)

Cas	Effet
Le symbole existe dans un enclos	la valeur y est remplacée
Il n'existe nulle part	il est créé au niveau global

« *For most users `<<-` creates a global variable ... Only when `<<-` has been used in a function that was returned as the value of another function will the special behavior described here occur.* »

Dans une fonction ordinaire, `<<-` **pollue l'espace de travail**. Son intérêt réel est **dans une fermeture**.

« *Lexical scope can also be used to give functions mutable state. ... We achieve this by creating the three functions within `account` and then returning a list containing them. ... Because these functions are defined in an environment which contains `total`, they will have access to its value.* » (§10.7)

Le mécanisme. L'appel `open.account(100)` crée un environnement contenant `total`. Les trois fonctions y sont **définies**, donc le **capturent**. L'appel se termine, mais l'environnement **survit** tant que les fonctions le référencent.

Pourquoi pas `<-`. Une assignation ordinaire est « *local and temporary and lost after exit* » (§10.5, fiche 309) : le solde reviendrait à sa valeur initiale au retrait suivant. `<<-` **écrit dans l'enclos**, donc là où `total` vit réellement.

Deux comptes indépendants : `ross <- open.account(100)` et `robert <- open.account(200)` déclenchent **deux appels**, donc **deux environnements** — d'où `70` et `200` aux appels de `balance()`.

```
withdraw = function(amount) {
  if (amount > total)
    stop("You don't have that much money!\n")
  total <<- total - amount
}
```

```
cat(amount, "withdrawn. Your balance is", total, "\n\n")
}
```

Parce que `stop()` **interrompt l'exécution** : la ligne `total <- total - amount` **n'est jamais atteinte**. Le solde reste donc **intact** après un `ross$withdraw(500)` .

Si le garde-fou venait après, `total` serait déjà devenu négatif — et `<-` **l'aurait écrit dans l'enclos**, donc de façon **permanente**. L'ordre des lignes est ici **une garantie d'intégrité**, pas un choix de style.

Même logique dans `deposit` : `if (amount <= 0) stop("Deposits must be positive!\n")` précède l'écriture.

« ***Making variables defined higher up the call stack available is called dynamic scope. The binding for a variable is then determined by the most recent (in time) definition of the variable. This contradicts the default scoping rules in R, which use the bindings in the environment in which the function was defined (lexical scope).*** » (§3.5.3)

« ***Some functions, particularly those that use and manipulate model formulas, need to simulate dynamic scope by directly accessing the call stack.*** »

C'est l'explication technique de `lm(y ~ x, data = dummy)` (fiche 313) : la formule doit être évaluée **là où elle a été écrite**, c'est-à-dire chez l'appelant.

Les outils : la famille `sys.*` — `sys.call` , `sys.frame` , `sys.function` , `sys.parent` , `sys.calls` , `sys.frames` , `sys.parents` , `sys.on.exit` , `sys.status` — et `parent.frame` .

« ***Packages which have a namespace have a different search path. When a search for an R object is started from an object in such a package, the package itself is searched first, then its imports, then the base namespace and finally the global environment and the rest of the regular search path.*** » (§3.5.4)

« ***The effect is that references to other objects in the same package will be resolved to the package, and objects cannot be masked by objects of the same name in the global environment or in other packages.*** »

C'est la solution du problème de masquage de la fiche 306. Redéfinir `mean` dans votre espace de travail **ne change rien** au fonctionnement interne d'un paquet à espace de noms :

il trouve **son** `mean` avant le vôtre.

Sans espace de noms, l'ordre est celui de `search()` — et le premier trouvé gagne.

*« Thus, **the sequence in which files are executed is : `Rprofile.site` , the user profile, `.RData` and then `.First()` . A definition in later files will mask definitions in earlier files.** »* (§10.8)

Étape	Détail
<code>Rprofile.site</code>	via <code>R_PROFILE</code> ; sinon <code>etc/Rprofile.site</code> du répertoire d'installation
profil utilisateur	<code>.Rprofile</code> du répertoire courant , sinon du répertoire personnel ; <code>R_PROFILE_USER</code> l'emporte
<code>.RData</code>	l'espace de travail sauvegardé
<code>.First()</code>	« <i>automatically performed at the beginning of an R session</i> »

Le dernier chargé l'emporte : un `.RData` peut donc écraser une définition du profil de site, **silencieusement** — d'autant que `ls()` ne montre pas les noms commençant par un point (fiche 300).

`.Rprofile` « *gives individual users control over their workspace and **allows for different startup procedures in different working directories*** ». Et `.Last()` « *is (normally) executed at the very end of the session* ».

« *Environments may be assigned to variables or be contained in other objects. **However, notice that they are not standard objects — in particular, they are not copied on assignment.*** » (§3.5.2)

Et (§2.1.10, fiche 303) : « ***if you assign the same environment to several symbols and change one, the others will change too*** ».

La conséquence ici est exactement ce qui rend `open.account` possible. Les trois fermetures de `ross` **partagent** le même environnement — elles n'en ont pas chacune une

copie. C'est pourquoi un dépôt fait par `deposit` est visible par `balance` .

Si les environnements étaient copiés comme le reste (fiche 302), l'état mutable serait **impossible**.

« *when a variable is requested inside a function, it is **first sought in the evaluation environment**, then in **the enclosure, the enclosure of the enclosure**, etc. ; once **the global environment or the environment of a package** is reached, the search continues **up the search path** to the environment of **the base package**. If the variable is not found there, the search will proceed next to **the empty environment, and will fail**.* » (§3.5.2)

environnement d'évaluation

```
-> enclos (= environnement de DEFINITION de la fonction)
-> enclos de l'enclos ...
-> .GlobalEnv ou environnement d'un paquet
-> chemin de recherche (attaches, Autoloads)
-> package:base
-> environnement VIDE -> ECHEC
```

L'**enclos** est l'environnement **de définition**, pas celui de l'appelant — c'est toute la portée lexicale.

Flashcards

Question	Réponse
Comment le cours qualifie-t-il le §10.7 ?	« <i>one of the major differences between S-Plus and R</i> »
Les trois classes de symboles ?	Formels, locaux, libres
Qu'est-ce qu'un paramètre formel ?	Un symbole de la liste d'arguments
Une variable locale ?	Définie par une expression du corps
Une variable libre ?	Ni formelle ni locale
Que devient une variable libre assignée ?	Elle devient locale
Comment R définit-il ses variables ?	Il n'a pas de définition formelle — elles s'utilisent
Comment une locale se reconnaît-elle ?	Par sa présence à gauche d'une assignation
Le nom de la règle de portée de R ?	La portée lexicale
Où cherche-t-elle les variables libres ?	Dans l'environnement de création
Et S-Plus ?	Dans une variable globale — portée statique
Que vaut <code>cube(2)</code> en R ?	8
En S-Plus avec <code>n <- 3</code> ?	18
Et sans <code>n</code> global en S-Plus ?	Une erreur
Qu'est-ce qu'un <i>frame</i> ?	Un ensemble de variables
Qu'est-ce qu'un <i>environment</i> ?	Un emboîtement de frames
Autre formulation ?	Frame le plus interne + enclos
Qu'est-ce qu'une fermeture ?	Le code + son environnement de création
Ses trois composantes ?	<code>formals</code> , <code>body</code> , <code>environment</code>
Sont-elles modifiables ?	Oui — à gauche d'une assignation
L'enclos d'un appel, c'est ?	L'environnement de la fermeture

Question	Réponse
Est-ce celui de l'appelant ?	« <i>not necessarily</i> »
Les environnements sont-ils copiés ?	Non
Où commence la recherche d'un symbole ?	L'environnement d'évaluation
Puis ?	L' enclos , puis l'enclos de l'enclos
Puis ?	Le chemin de recherche jusqu'à <code>base</code>
Et enfin ?	L' environnement vide — échec
Que fait <code><<-</code> s'il trouve le symbole ?	Il remplace la valeur dans cet enclos
S'il ne le trouve pas ?	Il le crée au niveau global
Que fait-il pour la plupart des usages ?	Il crée une variable globale
Quand a-t-il son comportement intéressant ?	Dans une fonction renvoyée par une autre
À quoi sert la portée lexicale, au-delà de la portée ?	À donner aux fonctions un état mutable
Le patron de l'état mutable ?	Fabrique + fonctions internes + <code><<-</code> + liste
Deux appels de la fabrique donnent ?	Deux états indépendants
Pourquoi <code>stop()</code> avant l'assignation ?	Pour que l'état ne soit pas modifié
Qu'est-ce que la portée dynamique ?	Chercher plus haut dans la pile d'appels
Qu'est-ce qui détermine la liaison ?	La définition la plus récente dans le temps
Qui en a besoin en R ?	Les fonctions de formules de modèle
Comment y accède-t-on ?	La famille <code>sys.*</code> et <code>parent.frame</code>
Que rend <code>sys.call</code> ?	L'appel du contexte indiqué
Que rend <code>sys.parent</code> ?	Le parent de l'invocation courante
Que fait <code>sys.on.exit</code> ?	Pose une fonction à exécuter à la sortie du contexte
Qu'est-ce que l'environnement global ?	La racine de l'espace de travail

Question	Réponse
Où va un objet assigné en ligne de commande ?	Dans l' environnement global
Premier élément du chemin de recherche ?	<code>.GlobalEnv</code>
Dernier ?	Le paquet <code>base</code>
À quoi sert <code>AutoLoads</code> ?	Aux objets chargés à la demande
Comment insère-t-on un environnement ?	<code>attach</code> ou <code>library</code>
L'ordre de recherche dans un paquet à espace de noms ?	Le paquet, ses imports , <code>base</code> , puis le global
Que cela empêche-t-il ?	Le masquage par un objet de même nom
Les trois moyens de personnaliser l'environnement ?	Fichier de site , fichier par répertoire , <code>.First / .Last</code>
Où est le fichier de site ?	<code>R_PROFILE</code> , sinon <code>etc/Rprofile.site</code>
Où est le profil personnel ?	<code>.Rprofile</code> — courant, puis personnel
Quelle variable l'emporte sur lui ?	<code>R_PROFILE_USER</code>
L'ordre d'exécution ?	<code>Rprofile.site</code> → profil utilisateur → <code>.RData</code> → <code>.First()</code>
Quelle règle de priorité ?	« <i>later files will mask definitions in earlier files</i> »
Quand <code>.Last()</code> est-elle exécutée ?	À la toute fin de la session
D'où <code>.First()</code> peut-elle venir ?	Des profils ou de <code>.RData</code>